IC-307D: DATA STRUCTURES & ALGORITHMS LABORATORY

Laboratory Summary & System Architecture Presentation

Comprehensive Pure-Python Implementations from First Principles (Units 1 through 7)

Course: IC-307D Data Structures & Algorithms Lab	**Department:** Department of Computer Science & Engineering
Student Identity / Roll: Jeet Verma / IC-2K23-46	**Institution:** International Institute of Professional Studies (IIPS), DAVV, Indore, MP
Academic Term: Academic Year 2026–2027	**Units Covered:** Units 1 through 7 (Arrays to Graphs)
Implementation: Python 3.8+ (Zero 3rd-Party DS Libraries)	**Verification:** Automated Self-Test Assertion Harness

Introduction & Laboratory Objectives

Pedagogical Philosophy & Core Mandate

- **First-Principles Implementation**: No reliance on third-party algorithmic frameworks or Python container shortcuts (collections.deque, queue.Queue, heapq, bisect, eval()).
- **Authentic Invariant Modeling**: Pointer dereferencing, manual memory shifting, binary tree self-balancing, and graph relaxation invariants implemented manually.
- **Defensive Architecture**: Strict input validation, dimension compatibility checking, singular matrix detection, and exception raising on boundary violations.

Course Learning Outcomes (COs)

- **CO1: Linear Data Structures**: Contiguous 1D/2D arrays, sparse coordinate lists, singly/doubly/circular linked lists, static/dynamic stacks, and linear/circular/priority queues.
- **CO2: Sorting, Searching, and Hashing**: Comparison lower bounds ($\Omega(n \log n)$), 7 sorting algorithms, binary search, and polynomial hash collision resolution.
- **CO3: Non-Linear Hierarchical & Relational Structures**: Binary trees, self-balancing AVL trees, expression trees, graph representations, BFS, DFS, and Dijkstra's algorithm.

Computational Environment & Standards

- **Operating Environment**: Linux (Ubuntu/Debian x86_64 POSIX environment).
- **Programming Language**: Python 3.8+ (Standard Library only).
- **Quality Assurance**: Automated self-tests invoked via --test flag across all programs.
- **Asymptotic Discipline**: Formal verification of best, average, and worst-case time complexities and auxiliary space bounds.

Key Metrics Evaluated Across Units

- **Time Complexity**: Big-O growth rate modeling execution latency.
- **Auxiliary Space Complexity**: Temporary operational memory footprint.
- **Memory Layout**: Cache-friendly contiguous buffers vs. dynamic heap node pointers.
- **Structural Invariants**: LIFO, FIFO, BST ordering, and AVL balance factors.

LABORATORY STRUCTURE & PROGRAM INVENTORY

Architectural Overview of the Seven Units

Unit	Topic Focus	Programs	Core Data Structures Implemented	Primary Applications
Unit 1	Arrays & Matrices	**4 Progs**	1D Contiguous Array, 2D Grid, Sparse 3-Tuple	Matrix Calculator, Sparsity Optimization
Unit 2	Sorting & Searching	**11 Progs**	7 Sorts, Binary Search, Hash Tables (Chain/Probe)	Student Manager, Search Engine Sim
Unit 3	Linked Lists	**6 Progs**	Singly, Doubly, and Circular Linked Lists	Polynomial Arithmetic, Text Buffer, Playlist
Unit 4	Stacks	**8 Progs**	Static Array Stack, Dynamic Linked Stack	Shunting-Yard Infix/Postfix, Undo/Redo Engine
Unit 5	Queues	**8 Progs**	Linear, Circular, Deque, Min-Heap Priority Queue	Round Robin CPU Scheduling, Customer Service
Unit 6	Trees	**7 Progs**	Binary Tree, BST, Self-Balancing AVL, Expression Tree	File System Simulator, Compiler Symbol Table
Unit 7	Graphs	**9 Progs**	Adjacency Matrix, Adjacency List, Dijkstra Heap	Social Network, 2D Maze Solver, Course DAG

Total Repository Scope: *53 modular programs across 7 units, comprehensively covering linear and non-linear paradigms from first principles with full algorithmic verification.*

Arrays, Sorting, and Searching Algorithms

Key Invariants & Implementations

- **1D Array Shifting**: Insertions and deletions at index *k* require shifting *n - k* elements (*O(n)* time, *O(1)* space).
- **In-Place Rotation**: Reversal algorithm rotates array in *O(n)* time and *O(1)* space via 3 subarray reversals.
- **Sparse Matrix Compression**: Canonical 3-tuple format [r, c, v] achieves space savings when non-zeros $K < (M \cdot N - 3)/3 \approx 33.3\%$.
- **Comparison Lower Bound**: Decision tree model proves $\Omega(n \log n)$ comparisons required for general sorting.
- **Divide-and-Conquer Sorting**: Merge Sort guarantees *O(n log n)* stable sort; Quick Sort achieves optimal in-place partitioning with median-of-3 pivot.
- **Non-Comparison Radix Sort**: Bypasses $\Omega(n \log n)$ lower bound in $O(d \cdot (n + b))$ time via digit passes.
- **Hash Tables**: Polynomial rolling hash with Separate Chaining vs Linear Probing for average *O(1)* retrieval.

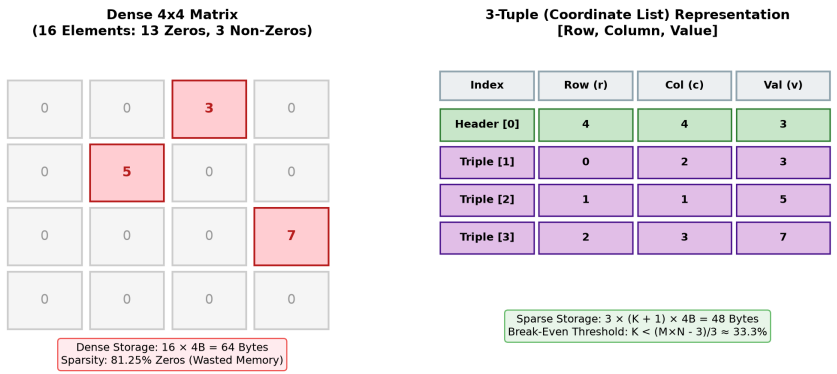


Figure 4.1: Dense 4x4 vs. 3-Tuple Coordinate List Compression

Sorting Complexity Quick Reference:

• Bubble/Selection/Insertion: $O(n^2)$ worst case
• Merge / Heap Sort: Guaranteed O(n log n)
• Quick Sort: O(n log n) average, $O(n^2)$ degenerate
• Binary Search: O(log n) bisection over ordered arrays

Linked Lists: Singly, Doubly, and Circular

Structural Topologies & Invariants

- **Singly Linked List (SLL)**: Unidirectional next pointer. *O(1)* head insertion; *O(n)* sequential traversal for index access.
- **Doubly Linked List (DLL)**: Bidirectional prev and next references with dual head and tail pointers. Enables *O(1)* operations at both ends and reverse traversal.
- **DLL Invariant Symmetry**: Enforces node.next.prev == node and node.prev.next == node.
- **Circular Linked List (CLL)**: Closed ring topology where tail.next == head. No None terminator; cycle-safe traversal loops.
- **In-Place Reversal**: SLL reversed via 3-pointer swapping (prev, curr, next); DLL reversed by swapping prev and next links for every node in *O(n)* time.
- **Polynomial Arithmetic**: Single-variable terms sorted in descending exponent order; two-pointer merge addition runs in *O(n + m)*.
- **Line Text Buffer**: DLL line modeling allows localized editing at cursor in *O(1)* without shifting large memory arrays.

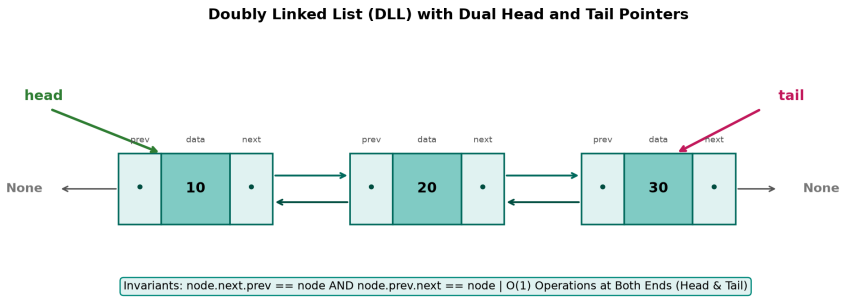


Figure 5.1: Doubly Linked List Architecture with Dual Pointers

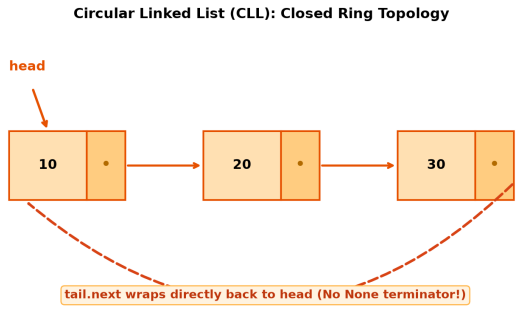


Figure 5.2: Circular Linked List Ring Topology

Stacks and Queues: LIFO, FIFO, and Priority Heaps

Operational Disciplines & Mechanics

- **Stack (LIFO Discipline)**: Push, pop, and peek constrained to top in *O(1)* time. Static buffer detects overflow/underflow; dynamic linked stack scales organically.
- **Shunting-Yard Algorithm**: Stack-based operator precedence and associativity parsing converts ambiguous infix expressions to parenthesis-free postfix in *O(N)* time.
- **Dual-Stack Undo/Redo**: Command/Memento pattern using undo_stack and redo_stack with history branching invalidation.
- **Queue False Overflow**: Linear arrays trigger overflow when *rear = cap - 1* despite vacated front slots.
- **Circular Queue Modulo Arithmetic**: Pointer wraparound via (ptr + 1) % cap reuses memory in strict *O(1)* latency.
- **Priority Queue (Binary Min-Heap)**: Array mapping Parent(i) = ■(i-1)/2■; sift_up and sift_down achieve optimal *O(log n)* push and pop.
- **Round Robin Scheduling**: Preemptive multi-process queue simulation calculating AT, BT, CT, TAT, and WT.

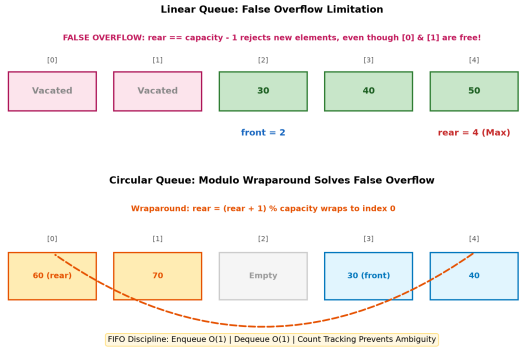


Figure 6.1: Linear False Overflow vs. Circular Modulo Wraparound

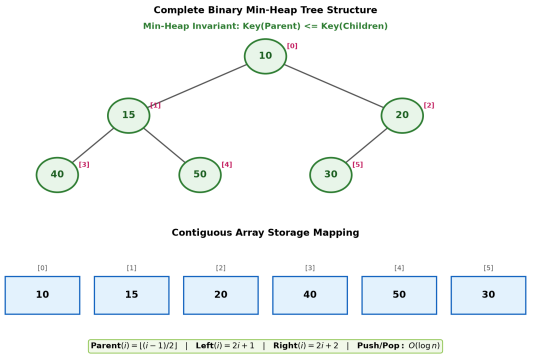


Figure 6.2: Binary Min-Heap Tree Structure and Array Mapping

Trees: Binary Search Trees & Self-Balancing AVL Trees

Hierarchical Invariants & Operations

- **Tree Traversals**: Preorder (Root-L-R), Inorder (L-Root-R), Postorder (L-R-Root) in *O(n)* time. Iterative traversals use explicit LIFO stacks to bypass Python's 1000-frame recursion limit.
- **BST Invariant**: For every node *X*: Left(X) < Key(X) < Right(X). Inorder traversal yields strictly monotonic sorted output.
- **BST Deletion 3 Cases**: Case 1 (Leaf): unlink node; Case 2 (1 child): promote child; Case 3 (2 children): replace with **inorder successor** (minimum in right subtree).
- **AVL Self-Balancing Invariant**: Balance factor BF = h_L - h_R ∈ {-1, 0, 1}. Guaranteed height $h \leq 1.44 \log(n)$.
- **The Four AVL Rotations**: Left-Left (LL → Right Rotate), Right-Right (RR → Left Rotate), Left-Right (LR → Double Rotate), Right-Left (RL → Double Rotate). Restores balance in *O(1)* time.
- **Expression Tree**: Constructed from postfix tokens; evaluates arithmetic expressions bottom-up recursively.
- **Compiler Symbol Table**: BST tracking identifier data types, scope depths, and source line numbers.

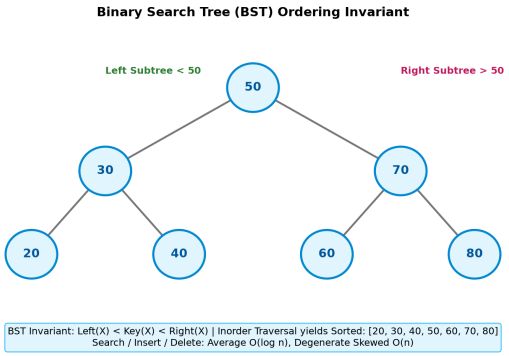


Figure 7.1: Binary Search Tree Ordering Invariant

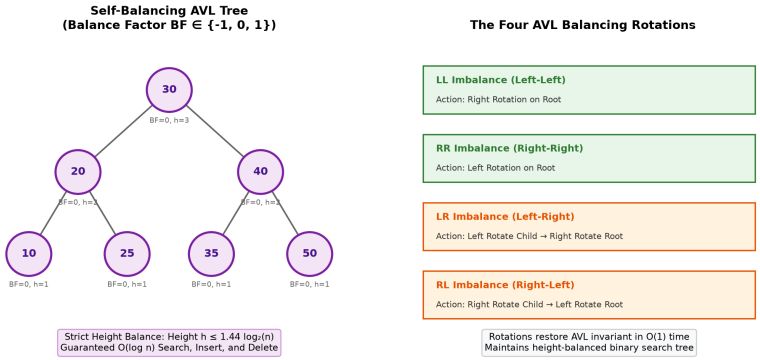


Figure 7.2: Self-Balancing AVL Tree and Four Rotations

Graphs: Representations, Traversals & Shortest Paths

Network Topologies & Algorithms

- **Adjacency Matrix**: $V \times V$ 2D grid. $O(1)$ edge existence query; $O(V^2)$ memory. Optimal for dense interconnection networks.
- **Adjacency List**: Associative map of edge nodes. Consumes optimal $O(V + E)$ memory for sparse real-world networks.
- **Breadth-First Search (BFS)**: Iterative level-by-level queue exploration in $O(V + E)$. Guarantees finding shortest paths in unweighted graphs.
- **Depth-First Search (DFS)**: Deep path exploration via call stack or explicit LIFO stack in $O(V + E)$. Used for cycle detection and backtracking.
- **Dijkstra's Algorithm**: Single-source shortest path on weighted graphs using custom Binary Min-Heap Priority Queue. Operates in $O((V + E) \log V)$ time via edge relaxation: dist[v] = min(dist[v], dist[u] + w(u,v)).
- **Social Network Simulator**: Mutual friends and 2-hop recommendations.
- **Course Prerequisites (DAG)**: Kahn's Topological Sort computes in-degrees to schedule courses and detect circular dependency deadlocks.

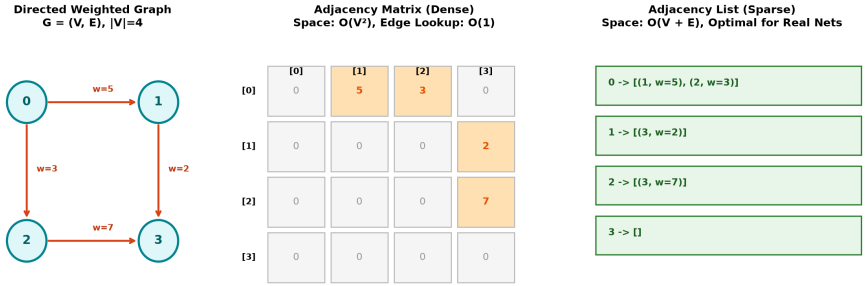


Figure 8.1: Graph, Adjacency Matrix, and Adjacency List

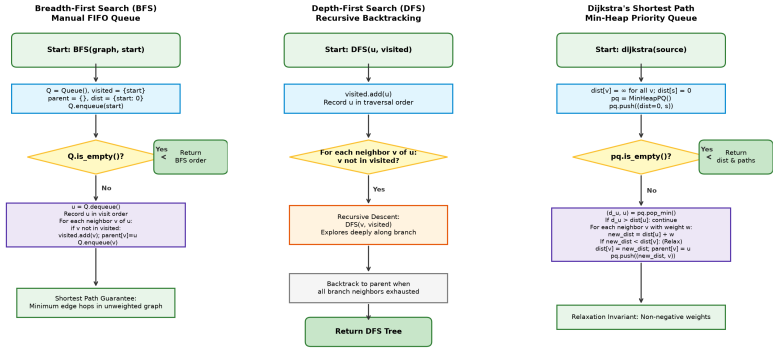


Figure 8.2: BFS, DFS, and Dijkstra's Shortest Path Logic

THEORETICAL EVALUATION & SYNTHESIS

Algorithmic Complexity Comparisons

Category / Algorithm	Best Case	Average Case	Worst Case	Aux. Space	Primary Architectural Invariant
Bubble Sort (Optimized)	O(n)	O(n²)	O(n²)	O(1)	Stable. Early-exit flag halts after 1 pass on sorted data.
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Stable. Divide-and-conquer bisection tree depth log n.
Quick Sort (Median-3)	O(n log n)	O(n log n)	O(n²)	O(log n)	Unstable. In-place partitioning around median-of-3 pivot.
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	Unstable. In-place binary max heap with sift_down.
Binary Search (Iterative)	O(1)	O(log n)	O(log n)	O(1)	Bisection of sorted contiguous array; zero extra memory.
Hash Table (Separate Chain)	O(1)	O(1)	O(n)	O(m + n)	Polynomial rolling hash with linked collision buckets.
Binary Search Tree (BST)	O(1)	O(log n)	O(n)	O(h)	Unbalanced: skews to linear chain on pre-sorted input.
AVL Tree (Self-Balancing)	O(1)	O(log n)	O(log n)	O(log n)	Strict balance |BF| ≤ 1; height bounded by 1.44 log■(n).
Graph BFS / DFS	O(V + E)	O(V + E)	O(V + E)	O(V)	Linear graph exploration; BFS yields unweighted shortest hops.
Dijkstra (Binary Heap)	O((V+E)log V)	O((V+E)log V)	O((V+E)log V)	O(V)	Greedy relaxation with min-heap (non-negative weights).

Theoretical vs. Empirical Note: *Quick Sort exhibits smaller constant factors and better cache locality than Merge Sort despite identical O(n log n) average bounds, while AVL Trees eliminate BST worst-case degradation.*

REAL-WORLD SYSTEMS INTEGRATION

Practical Applications and Industry Use Cases

Operating Systems & Kernel Architecture

- **CPU Process Dispatching**: Round Robin scheduling uses circular queues to cycle processes fairly across time quantums without false overflow.
- **Interrupt & Event Queues**: Priority queues backed by binary heaps schedule timer interrupts and high-priority kernel threads.
- **File Systems**: Operating system directory namespaces are modeled as general *N*-ary hierarchical trees (implemented in Program 6).

Compilers, Parsers & Language Runtimes

- **Arithmetic Syntax Transformation**: Shunting-Yard stack algorithm translates source expressions into postfix bytecode.
- **Abstract Syntax Trees (ASTs)**: Expression trees parse algebraic precedence and evaluate intermediate code bottom-up.
- **Symbol Tables**: Compiler variable declarations, type signatures, and scopes managed via Binary Search Trees.

Databases, Networking & Navigation

- **Database Indexing**: Self-balancing AVL trees and B-Trees maintain ordered key indices guaranteeing *O(log n)* records lookup.
- **Network Routing Protocols**: OSPF and IS-IS link-state routers use Dijkstra's Algorithm to calculate least-cost packet paths across the global Internet.
- **Spatial Navigation & Robotics**: Breadth-First Search (BFS) navigates grid tilemaps and mazes to discover optimal shortest escape routes.

Interactive Software Systems

- **Document Text Buffers**: Doubly linked lists power text editor buffers, allowing instant line editing without shifting arrays.
- **Transaction Rollback (Undo/Redo)**: Dual-stack engines enable user action undoing and redoing with branch invalidation.
- **Dependency Build Schedulers**: DAG topological sorting detects circular dependency deadlocks in software build systems.

PEDAGOGICAL & TECHNICAL COMPETENCIES

Learning Outcomes and Engineering Insights

Algorithmic Mastery & Problem Solving

- **Memory Contiguity vs Dynamic Pointers**: Experienced how contiguous arrays provide constant-time indexing but incur expensive shifting overheads, while linked structures provide *O(1)* pointer insertions at the expense of sequential access.
- **Asymptotic Evaluation Fluency**: Gained deep ability to deduce mathematical worst, average, and best-case complexities from first principles.
- **Divide-and-Conquer & In-Place Design**: Implemented high-performance algorithms (Merge Sort, Quick Sort, Reversal Rotation, Heap Sort) balancing time efficiency and memory footprint.
- **Self-Balancing Invariants**: Mastered AVL tree balance factor tracking and rotational mechanics restoring logarithmic height bounds.

Professional Software Engineering Discipline

- **First-Principles Implementation**: Built complex data structures without relying on library abstractions, mastering the actual low-level pointer dereferencing and buffer management.
- **Defensive Programming & Robustness**: Enforced strict validation: matrix rectangularity, empty structure underflow detection, circular list termination, and singular matrix handling.
- **Recursion vs Iteration Optimization**: Successfully converted recursive algorithms to explicit stack-based iteration to overcome production call-stack limits.
- **Automated Verification**: Verified all 53 programs with comprehensive unit self-tests covering standard and boundary edge cases.

SUMMARY & CLOSING REMARKS

Conclusion & Synthesis

Laboratory Work Summary

- **Seven Units Fully Implemented**: Successfully developed, tested, and analyzed 53 foundational DSA programs across arrays, sorting, searching, linked lists, stacks, queues, trees, and graphs.
- **Zero External Dependencies**: Maintained pure Python 3.8+ standard library implementation, ensuring maximum transparency into memory layout and pointer semantics.
- **Complete Documentation Suite**: Created 11 data structure diagrams, 8 algorithm flowcharts, a comprehensive complexity analysis reference, and this academic presentation.

Core Architectural Philosophy

"The appropriate choice of data structure transforms computationally intractable problems into linear or logarithmic execution, fundamentally determining software performance and scalability."

Project Artifacts Generated

- **docs/diagrams/**: 11 PlantUML (.puml) source files + 11 publication PNG diagrams.
- **docs/algorithm-flowcharts/**: 8 PlantUML (.puml) source files + 8 publication PNG flowcharts.
- **docs/complexity-analysis.pdf**: Rigorous unit-wise mathematical breakdown and comparative analysis.
- **docs/final-report.pdf**: Academic laboratory manual documenting all programs, architectures, and diagrams.
- **docs/presentation.pdf**: Concise visual slide deck summarizing the laboratory achievements.

Thank you. Questions & Discussion.